Análise de Desempenho de Algoritmos de Ordenação em Listas Encadeadas para Ranking Dinâmico de Atletas

Grupo 5 — Disciplina: Estrutura de Dados
Bacharelado em Engenharia de Software — Prof. Weskley Mauricio
Crato, Ceará, Brasil

ABSTRACT

Este trabalho analisa o desempenho de cinco algoritmos de ordenação — Bubble Sort, Insertion Sort, Selection Sort, Quick Sort e Merge Sort — aplicados a um sistema de ranking de atletas em corridas de rua. Foram testados três volumes de dados ($N = 100$, $N = 1000$ e $N = 10000$), três tipos de entrada e duas estruturas de dados: lista encadeada dinâmica e lista encadeada estática. Cada experimento foi repetido 100 vezes para garantir resultados confiáveis. Os resultados mostraram que o Merge Sort é o mais adequado para esse tipo de aplicação, pois garante complexidade $O(n \log n)$ em todos os casos e realiza o menor número de comparações entre todos os algoritmos testados.

Palavras-chave—*Algoritmos de Ordenação, Estruturas de Dados, Lista Encadeada, Análise de Desempenho, Ranking de Atletas.*

I. INTRODUÇÃO

Imagine um aplicativo que mostra o ranking ao vivo de uma corrida de rua. A cada minuto, atletas terminam a prova, outros abandonam e novos tempos são registrados. A lista precisa ser reordenada constantemente, e o usuário pode querer ver os corredores por pace, por tempo total ou por número de dorsal.

Esse cenário motivou este trabalho: queremos descobrir qual algoritmo de ordenação é o mais adequado para esse tipo de situação, onde os dados mudam o tempo todo e a ordenação precisa ser rápida e previsível.

Cada algoritmo tem características diferentes. Alguns são simples de implementar, mas ficam lentos quando a lista cresce. Outros são mais complexos, porém mantêm um bom desempenho independente do tamanho da lista [2]. Para decidir qual é o melhor, implementamos os cinco algoritmos pedidos, medimos o tempo de execução e o número de comparações, e comparamos os resultados.

Além dos algoritmos, também analisamos o impacto da estrutura de dados: testamos os algoritmos tanto em lista encadeada dinâmica quanto em lista encadeada estática, para ver se a estrutura faz diferença no desempenho final [1].

II. DESENVOLVIMENTO

A. O Caso Real: Ranking de Corrida

Para o nosso sistema, cada atleta é representado por uma estrutura com as seguintes informações: número do dorsal (que serve como identificador único), nome completo, tempo decorrido em segundos, pace (ritmo em min/km), distância percorrida, posição no ranking e status, que pode ser *Ativo*, *Finalizado* ou *DNF* (Did Not Finish, quando o atleta abandona a prova).

Durante a simulação, novos atletas podem entrar na lista e outros podem ser removidos, o que representa bem o que acontece numa corrida de verdade. A lista pode ser reordenada por qualquer um dos campos, dependendo do que o usuário quiser visualizar.

B. As Estruturas de Dados

Lista Encadeada Dinâmica

Nessa versão, cada nó é criado com `malloc` quando um novo atleta entra, e liberado com `free` quando ele sai. A inserção no início é rápida ($O(1)$), mas buscar ou remover um atleta exige percorrer a lista ($O(n)$). A vantagem é que não há limite de tamanho; a desvantagem é o custo de cada alocação.

Lista Encadeada Estática

Aqui, ao invés de pedir memória ao sistema a cada inserção, usamos um vetor fixo de 10.010 posições criado de uma vez só. Os "ponteiros" são na verdade índices desse vetor, e os espaços livres são gerenciados manualmente por uma lista interna. Isso elimina o overhead das alocações, mas limita o número máximo de atletas.

C. O Algoritmo Escolhido: Merge Sort

Após comparar todos os algoritmos, escolhemos o **Merge Sort** como o mais adequado para o ranking de atletas. Os motivos estão detalhados na Seção 3, mas em resumo: ele é o único que garante bom desempenho *sempre*, sem surpresas.

Como o Merge Sort funciona

A ideia é simples: divide a lista ao meio repetidamente até ter pares de elementos, depois vai juntando (intercalando) esses pares em ordem. O pseudocódigo abaixo descreve o processo:

```
função MergeSort(v, n, cmp)
if n ≤ 1 then
    return
end if
aux ← vetor auxiliar de tamanho n
MergeSortRec(v, aux, 0, n−1, cmp)

função MergeSortRec(v, aux, esq, dir, cmp)
if esq ≥ dir then
    return
end if
meio ← (esq + dir) / 2
MergeSortRec(v, aux, esq, meio, cmp)
MergeSortRec(v, aux, meio+1, dir, cmp)
Intercala(v, aux, esq, meio, dir, cmp)

função Intercala(v, aux, esq, meio, dir, cmp)
i ← esq; j ← meio+1; k ← esq
while i ≤ meio and j ≤ dir do
    if cmp(v[i], v[j]) ≤ 0 then
        aux[k++] ← v[i++]
    else
        aux[k++] ← v[j++]
    end if
end while
while i ≤ meio do
    aux[k++] ← v[i++]
end while
while j ≤ dir do
    aux[k++] ← v[j++]
end while
for x = esq to dir do
    v[x] ← aux[x]
end for
```

Complexidade de todos os algoritmos

A Tabela 1 mostra a complexidade teórica dos cinco algoritmos implementados, nos três casos possíveis.

Table 1: Complexidade teórica dos algoritmos

Algoritmo	Melhor	Médio	Pior
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$

O ponto mais importante da tabela é a coluna do pior caso: o Merge Sort é o único que mantém $O(n \log n)$ mesmo na situação mais desfavorável. O Quick Sort, por exemplo, pode chegar a $O(n^2)$ em certos casos específicos. Para um ranking ao vivo, essa garantia faz toda a diferença.

III. RESULTADOS

A. Como fizemos os testes

O código foi implementado em C (padrão C11) e compilado com GCC 15.2.0. Medimos apenas o tempo do algoritmo de ordenação em si, sem contar o tempo de gerar os dados ou imprimir resultados. Cada teste foi repetido 100 vezes e usamos a média dos tempos — isso deixa os resultados mais confiáveis e as curvas dos gráficos mais suaves.

Para $N = 10000$, os algoritmos quadráticos foram testados 10 vezes por rodada porque cada execução demorava mais de 30 segundos, tornando 100 repetições inviável.

Geramos três tipos de entrada: **aleatória** (caso médio), **ordenada** (melhor caso para alguns algoritmos) e **inversamente ordenada** (pior caso para Bubble Sort, Insertion Sort e Quick Sort).

B. Quanto tempo cada algoritmo levou

A Fig. 1 mostra como o tempo cresce com o volume de dados. É possível ver claramente dois grupos: os algoritmos $O(n^2)$ sobem muito rápido, enquanto Quick Sort e Merge Sort crescem de forma muito mais controlada.

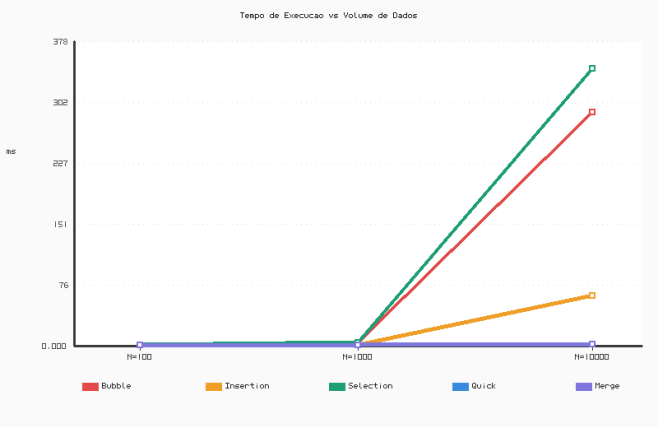

Figure 1: Tempo de execução versus volume de dados (Vetor, entrada aleatória).

A Tabela 2 coloca os números lado a lado. Para $N = 10000$, a diferença fica bem evidente: o Selection Sort demora 343 ms, enquanto o Merge Sort faz o mesmo trabalho em apenas 1,66 ms — mais de 200 vezes mais rápido.

Table 2: Tempo médio (ms) — Vetor, entrada aleatória

Algoritmo	N=100	N=1000	N=10000
Bubble Sort	0,0189	3,6618	290,0532
Insertion Sort	0,0060	0,5317	61,8852
Selection Sort	0,0300	3,3143	343,5657
Quick Sort	0,0027	0,0632	1,0884
Merge Sort	0,0033	0,0870	1,6648

C. Melhor, médio e pior caso

A Fig. 2 e a Tabela 3 mostram como cada algoritmo se comporta dependendo de como os dados chegam. Isso é importante porque numa corrida real não controlamos a ordem dos atletas.

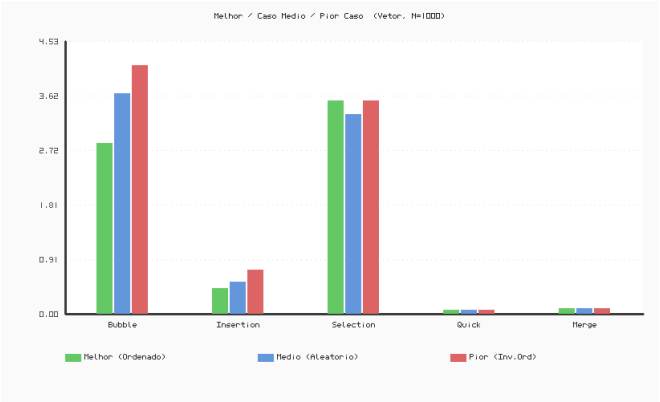

Figure 2: Melhor, caso médio e pior caso por algoritmo (Vetor, N=1000).

Table 3: Tempo médio (ms) por tipo de entrada — Vetor, N=1000

Algoritmo	Ordenado	Aleat.	Inv.
Bubble Sort	2,8357	3,6618	4,1169
Insertion Sort	0,4151	0,5317	0,7274
Selection Sort	3,5370	3,3143	3,5363
Quick Sort	0,0595	0,0632	0,0601
Merge Sort	0,0824	0,0870	0,0826

O que chama atenção no Merge Sort é a consistência: ele teve 0,0824 ms no melhor caso e 0,0870 ms no pior — uma variação mínima de apenas 0,0046 ms. O Quick Sort foi um pouco mais rápido no caso médio (0,0632 ms), mas o Merge Sort é mais previsível. Para um ranking que precisa ser atualizado em tempo real, isso importa.

D. O impacto da estrutura de dados

Será que a estrutura de dados (dinâmica ou estática) faz diferença no tempo de ordenação? A Fig. 3 e a Tabela 4 respondem essa pergunta para $N = 10000$.

A diferença entre as estruturas ficou menor do que esperávamos. A lista dinâmica foi apenas cerca de 2% mais

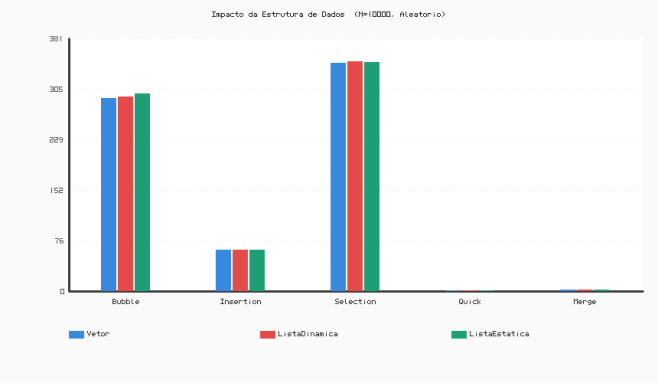

Figure 3: Impacto da estrutura de dados no desempenho (N=10000, aleatório).

Table 4: Impacto da estrutura — N=10000, aleatório (ms)

Algoritmo	Vetor	L. Din.	L. Est.
Quick Sort	1,0884	1,1100	0,9776
Merge Sort	1,6648	1,6964	1,6586

lenta que o vetor puro, provavelmente porque o tempo de ordenação domina o tempo total. Já a lista estática ficou levemente mais rápida que o vetor em alguns casos, o que faz sentido: seus elementos ficam todos juntos na memória, o que ajuda o processador.

E. Quantas comparações cada algoritmo faz

Além do tempo, também contamos quantas vezes cada algoritmo compara dois atletas entre si. Esse número independe do hardware e mostra melhor a eficiência teórica de cada abordagem.

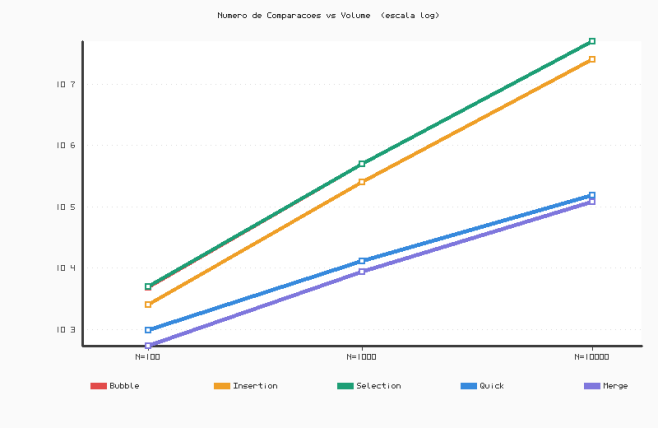

Figure 4: Número médio de comparações em escala logarítmica (Vetor, aleatório).

O Merge Sort foi o que fez menos comparações em todos os testes. Para $N = 10000$, ele comparou 120.403 pares, contra 49,9 milhões do Bubble Sort — uma diferença enorme. Vale notar que o Selection Sort sempre faz

Table 5: Média de comparações — Vetor, entrada aleatória

Algoritmo	N=100	N=1000	N=10000
Bubble Sort	4.859	499.490	49.990.050
Insertion Sort	2.515	252.742	25.043.884
Selection Sort	4.950	499.500	49.995.000
Quick Sort	960	13.124	152.035
Merge Sort	548	8.727	120.403

exatamente $n(n-1)/2$ comparações, não importa como os dados estão organizados.

F. Por que escolhemos o Merge Sort

Com base em tudo que medimos, o Merge Sort foi a escolha mais adequada para o ranking de atletas pelos seguintes motivos:

1. **Sempre** $O(n \log n)$: nunca vai surpreender com uma ordenação lenta, independente de como os atletas estiverem distribuídos.
2. **Menos comparações**: fez o menor número de comparações em todos os testes, inclusive comparado ao Quick Sort.
3. **Estabilidade**: atletas com o mesmo pace mantêm a ordem em que estavam antes — isso é importante para desempate.
4. **Resultados consistentes**: a diferença entre seu melhor e pior caso foi de apenas 0,0046 ms para $N = 1000$. Muito mais previsível.
5. **Funciona bem com todas as estruturas**: dá resultados parecidos tanto no vetor quanto nas duas listas.

O Quick Sort até foi mais rápido no caso médio (0,0632 ms contra 0,0870 ms, para $N = 1000$), mas pode degradar para $O(n^2)$ em alguns casos e teve mais variação entre as execuções. Para um ranking ao vivo, preferimos a estabilidade do Merge Sort.

IV. CONCLUSÃO

Neste trabalho, implementamos e comparamos cinco algoritmos de ordenação aplicados a um sistema de ranking dinâmico de atletas em corridas de rua. Testamos com três volumes de dados, três tipos de entrada e duas estruturas de dados diferentes.

Os resultados confirmaram o que a teoria já previa: os algoritmos $O(n^2)$ ficam muito lentos quando a lista cresce. Para $N = 10000$, o Selection Sort demorou 343 ms por ordenação — mais de 200 vezes mais que o Merge Sort. Entre os quadráticos, o Insertion Sort se saiu melhor quando a lista já estava quase ordenada, o que pode ser útil em cenários específicos.

O **Merge Sort** foi o algoritmo escolhido como ideal para o ranking de atletas. Ele garante $O(n \log n)$ em qualquer situação, faz o menor número de comparações, é estável e tem comportamento muito previsível. São exatamente as características que uma aplicação de ranking em tempo real precisa.

Sobre as estruturas de dados, a diferença de desempenho entre lista dinâmica e estática foi pequena para os algoritmos $O(n \log n)$. A lista estática teve uma ligeira vantagem, por manter os elementos contiguamente na memória. Para casos onde o número máximo de atletas é conhecido, ela seria a melhor opção.

Como próximos passos, seria interessante testar o Timsort — um híbrido entre Merge Sort e Insertion Sort usado em Python e Java — e avaliar o desempenho em cenários onde o ranking muda pouco entre uma atualização e outra.

REFERENCES

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA: Addison-Wesley, 1998.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.

[4] J. Bentley and M. D. McIlroy, "Engineering a sort function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.

[5] S. Wild and M. E. Nebel, "Average case analysis of Java 7's dual pivot quicksort," in *Proc. 20th European Symposium on Algorithms*, Ljubljana, 2012, pp. 825–836.